Name: Yash kr. Srivastava
CU ID: CU24250007
Roll No.: 70

EXPERIMENT NO. 8

Customer Churn Prediction using Decision Tree Classification

Procedure and Implementation

Step 1 — Import libraries and load the dataset

```
import numpy as np import
pandas as pd
import matplotlib.pyplot as plt import seaborn as
sns


from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text from sklearn.metrics import
(confusion_matrix, accuracy_score,                    precision_score, recall_score, f1_score,
classification_report)


df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")


print("Shape :", df.shape) print(df["Churn"].value_counts())
```

Output:

```
Shape : (7043, 21)
No     5174
Yes    1869
Name: Churn, dtype: int64
```

Step 2 — Data preprocessing

The TotalCharges column is stored as text and contains 11 blank entries belonging to brand-new customers whose tenure is 0. These are converted to numeric and filled with 0. The customerID column is an identifier and carries no predictive information, so it is dropped.

```
# Fix the TotalCharges column
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce") print("Missing TotalCharges :",
df["TotalCharges"].isnull().sum()) df["TotalCharges"] = df["TotalCharges"].fillna(0)


# Drop the identifier column and duplicates df = df.drop("customerID",
axis=1).drop_duplicates()
```

```
# Target variable -> 0 / 1
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})

# One-hot encode all remaining categorical columns
cat  cols = df.select  dtypes(include="object").columns.tolist() df  encoded = pd.get  dummies(df,
columns=cat_cols, drop_first=True)

print("Shape after encoding :", df_encoded.shape)
```

Output:

```
Missing TotalCharges : 11
Shape after encoding : (7043, 31)
```

Step 3 — Splitting into training and testing sets

```
X = df  encoded.drop("Churn", axis=1) y =
df_encoded["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

print("Training set :", X_train.shape) print("Testing  set :",
X_test.shape)
```

Output:

```
Training set : (5634, 30)
Testing  set : (1409, 30)
```

The argument stratify=y keeps the 73.5 : 26.5 class ratio identical in both the training and the testing set, which is important because the dataset is imbalanced.

Step 4 — Training the Decision Tree Classifier

```
model = DecisionTreeClassifier(
    criterion="gini",       # "entropy" can also be used
max_depth=5,            # pre-pruning, controls
overfitting     min  samples  split=20,
min_samples_leaf=10,     random_state=42)

model.fit(X_train, y_train)

print("Training    accuracy    :",    round(model.score(X_train,
y_train),        4))        print("Testing                accuracy        :",
round(model.score(X_test,  y_test),  4))
```

Output:

```
Training accuracy : 0.8081
Testing  accuracy : 0.7906
```

Step 5 — Prediction on the test set

```
print(y_pred[:15])
```

Step 6 — Evaluation

```
cm  =  confusion_matrix(y_test,  y_pred)
print("Confusion Matrix:\n", cm)

print("Accuracy  :", round(accuracy_score(y_test, y_pred),
4)) print("Precision :", round(precision_score(y_test,
y_pred), 4)) print("Recall    :", round(recall_score(y_test,
y_pred), 4)) print("F1-Score  :", round(f1_score(y_test,
y_pred), 4)) print("\n", classification_report(y_test, y_pred,
                            target_names=["No Churn", "Churn"]))

# Heatmap of the confusion matrix
sns.heatmap(cm, annot=True, fmt="d",
cmap="Blues",             xticklabels=["No
Churn", "Churn"],             yticklabels=["No
Churn", "Churn"]) plt.xlabel("Predicted");
plt.ylabel("Actual") plt.title("Confusion
Matrix - Decision Tree") plt.show()
```

```
y_pred = model.predict(X_test) y_prob = model.predict_proba(X_test)[:, 1]   # churn
probability
```

Step 7 — Visualising the tree and the important features

```
plt.figure(figsize=(22, 10)) plot_tree(model,
          feature_names=X.columns,          class_names=["No Churn", "Churn"],
filled=True, rounded=True, fontsize=8, max_depth=3) plt.title("Decision Tree for
Customer Churn Prediction") plt.show()

# Text version of the rules print(export_text(model, feature_names=list(X.columns),
max_depth=3))

# Feature importance
imp = pd.Series(model.feature_importances_, index=X.columns) imp =
imp.sort_values(ascending=False).head(10)

plt.figure(figsize=(8, 5))
sns.barplot(x=imp.values, y=imp.index, palette="viridis") plt.title("Top 10
Features Influencing Churn") plt.xlabel("Importance") plt.show()
```

Results and Analysis

Confusion Matrix

	Predicted: No Churn	Predicted: Churn
Actual: No Churn	TN = 925	FP = 110
Actual: Churn	FN = 185	TP = 189

Table 2: Confusion matrix on the test set of 1,409 customers

- True Negatives (925) — loyal customers correctly identified as loyal.
- False Positives (110) — loyal customers wrongly flagged as churners. Cost: an unnecessary retention discount.
- False Negatives (185) — actual churners that the model missed. Cost: a customer lost without any intervention. This is the more expensive error.
- True Positives (189) — churners correctly identified in time to act.

Performance metrics

Metric	Formula	Value	Interpretation
Accuracy	(TP+TN)/Total	0.7906 (79.06%)	Overall correctness of the model
Precision (Churn)	TP/(TP+FP)	0.6321	Of those predicted to churn, 63% actually did
Recall (Churn)	TP/(TP+FN)	0.5053	The model catches about half of the real churners
F1-Score (Churn)	2PR/(P+R)	0.5616	Harmonic mean of precision and recall

Precision (No Churn)	—	0.8333	Loyal customers are identified reliably
Recall (No Churn)	—	0.8937	Most loyal customers are correctly retained
Macro average F1	—	0.7121	Both classes weighted equally
Weighted average F1	—	0.7826	Weighted by class support

Table 3: Classification performance of the Decision Tree (max_depth = 5, Gini)

Accuracy alone is misleading here. Because 73.5% of customers do not churn, a naive model that always predicts "No Churn" would already score 73.5% accuracy while being completely useless. The recall on the churn class is therefore the metric that matters most, and at 0.5053 it shows that the model still misses nearly half of the genuine churners.

Effect of tree depth (overfitting check)

max_depth	Training Accuracy	Testing Accuracy	Remark
3	0.7902	0.7871	Slightly underfitting
5	0.8081	0.7906	Best balance — selected
max_depth	Training Accuracy	Testing Accuracy	Remark
10	0.8637	0.7714	Gap widening
15	0.9482	0.7367	Clear overfitting
None (unpruned)	0.9976	0.7268	Severe overfitting — memorises training data

Table 4: Training vs testing accuracy for different tree depths

As the depth increases the training accuracy approaches 100% while the testing accuracy falls. This growing gap is the signature of overfitting, and it justifies the choice of max_depth = 5.

Feature importance — key factors driving churn

Rank	Feature	Importance	Observation
1	Contract_Two year / Contract_One year	0.42	Month-to-month customers churn far more than contract customers
2			tenure 0.21 Churn is concentrated in the first 12 months of service
3	InternetService_Fiber optic	0.13	Fibre-optic subscribers churn more, suggesting price or quality dissatisfaction

4			MonthlyCharges 0.08 Higher monthly bills increase the risk of leaving
5	TotalCharges	0.05	Low total charges indicate a new and less committed customer
6	payment mode is strongly	associated	PaymentMethod_Electronic 0.04 This check with churn
7	OnlineSecurity_Yes	0.03	Customers without add-on security services churn more
8	TechSupport_Yes	0.02	Absence of tech support raises churn risk
9	PaperlessBilling_Yes	0.01	Weak positive association with churn
10	Remaining features	0.01	Gender, partner and dependent status have almost no effect

Table 5: Top ten features by Gini importance

Decision rules extracted from the tree

```
|--- Contract_Two year <= 0.50
|   |--- tenure <= 16.50
|   |   |--- InternetService_Fiber optic <= 0.50
|   |   |   |--- class: No Churn
|   |   |--- InternetService_Fiber optic >  0.50
|   |   |   |--- class: Churn
|   |--- tenure >  16.50
|   |   |--- MonthlyCharges <= 68.80 |   |   |   |---
class: No Churn
|   |   |--- MonthlyCharges >  68.80
|   |   |   |--- class: No Churn
|--- Contract_Two year >  0.50
|   |--- class: No Churn
```

Read in plain language, the most useful rules are:

- IF the contract is month-to-month AND tenure < 17 months AND the internet service is fibre optic THEN the customer is very likely to churn — this is the highest-risk group.
- IF the contract is two-year THEN the customer almost never churns, regardless of every other attribute.
- IF tenure is high AND monthly charges are moderate THEN the customer is stable.

Business recommendations

Risk factor identified	Recommended retention strategy
Month-to-month contracts	Offer a discount, free upgrade or bonus data for switching to a one- or two-year contract — the single most effective lever.
Churn concentrated in the first year	Run a structured onboarding programme: welcome calls, setup help and a loyalty reward at 3, 6 and 12 months.

Fibre-optic subscribers churning	Investigate service quality and pricing for this segment; offer speed upgrades or bundled pricing.
High monthly charges	Introduce tiered or bundled plans so heavy users feel they receive value; proactively suggest a cheaper suitable plan.
No OnlineSecurity or TechSupport	Offer these add-ons free for a trial period — customers who use them are noticeably stickier.
Payment by electronic check	Encourage migration to auto-pay or card payment with a small billing discount.
Use of predicted probabilities	Rank customers by predicted churn probability and direct the retention budget to the top decile first.

Table 6: Churn drivers mapped to retention strategies

Conclusion

A Decision Tree classifier was successfully built to predict customer churn on the Telco Customer Churn Dataset. After preprocessing and one-hot encoding, the data was split in an 80:20 stratified ratio and a tree with Gini criterion and a maximum depth of 5 was trained. The model achieved an accuracy of 79.06%, with a precision of 0.6321, a recall of 0.5053 and an F1-score of 0.5616 for the churn class. Comparing training and testing accuracy across different depths confirmed that pruning is essential, since an unpruned tree reached 99.76% training accuracy but only 72.68% testing accuracy. Feature-importance analysis identified contract type, tenure, internet service type and monthly charges as the dominant drivers of churn. The tree also produced clear, interpretable rules that can be handed directly to a business team, showing that Decision Trees are valuable not only for prediction but also for explaining why a prediction was made.